

Cluster Sizing

Strategy for Production

From 1 GB/day to 1 TB/day Growth

Data Volume → Node Configuration → Scaling Strategy

Core Sizing Principles

- **Partition Strategy:** Aim for 128–256 MB partitions for optimal parallel processing
- **Core-to-Data Ratio:** 1 GB/day $\approx$ 8–16 cores; 1 TB/day $\approx$ 150–250 cores
- **Memory per Core:** Maintain 4–8 GB per core to prevent spills and OOM errors
- **Autoscaling:** Use min/max worker bands for cost efficiency and SLA compliance

Starting Small: 1 GB/Day



Driver Node

CPU Cores

4

Memory

16–32 GB



Workers (Autoscaling)

Min Workers

1

Max Workers

2–3

Per Worker

4 cores, 16–32 GB

Scaling Up: 10 GB/Day

$10 \text{ GB} \div 128 \text{ MB} \approx 80 \text{ partitions} \rightarrow \text{Target } 8\text{--}16 \text{ cores minimum}$

Driver Node

CPU Cores

8

Memory

32 GB

If driver pressure detected

Workers (Autoscaling)

Min Workers

2

Max Workers

4–6

Per Worker

4 cores, 16–32 GB

Enterprise Scale: 1 TB/Day

1 TB ÷ 128 MB ≈ 7,800 partitions → Target 150–250 cores total

Driver Node

CPU Cores

16–32

Memory

64–128 GB

Large DAGs & many shuffles

Workers (Autoscaling)

Min Workers

8–10

Max Workers

20–25

Per Worker

8 cores, 32–64 GB

Growth Progression Over Time

 **1 GB/Day → 10 GB/Day**

Cores: 8–16 | Workers: 1–3 | Memory: 16–32 GB/worker



 **10 GB/Day → 100 GB/Day**

Cores: 32–64 | Workers: 4–8 | Memory: 32 GB/worker



 **100 GB/Day → 1 TB/Day**

Cores: 160–200 | Workers: 20–25 | Memory: 32–64 GB/worker

Continuous Tuning Strategy

- **Monitor Spark UI:** CPU %, memory %, shuffle spills, data skew
- **Adjust workers if:** Runtime exceeds SLA → increase max workers; Cores idle → decrease max workers
- **Memory spills?** → Increase instance size or reduce data per partition
- **Skew detected?** → Apply salting, AQE, or broadcast joins
- **Cost optimization:** Use spot instances for non-critical workloads, schedule off-peak runs

Quick Calculation Formula

Partition Count

`Partitions = Data Size (GB) / Partition Size (0.128 GB)`

`Example: 1000 GB / 0.128 = 7,800 partitions`

Core Requirement

`Target Cores  $\approx$  Partitions / 4 to 8`

`Example: 7,800 / 4  $\approx$  1,950 cores...  $\rightarrow$  Realistic: 160-200 cores with batching`

Memory Requirement

`Memory per Core = 4-8 GB`

`Total Memory = Cores  $\times$  Memory per Core`

`Example: 160 cores  $\times$  6 GB = 960 GB total cluster memory`

Production Best Practices

- **Use Job Clusters:** Auto-created/terminated, cost-effective for scheduled jobs
- **Enable Autoscaling:** Set min/max worker bands to match typical + spike loads
- **Memory-Optimized Instances:** For heavy joins/aggregations (prevent spills)
- **Regular Monitoring:** Check Databricks metrics and Spark UI for bottlenecks
- **Incremental Scaling:** Start small, add workers/cores as volume grows, measure impact